

Fast Primes - Cryptohack

Analysis

deskripsi chall:

I need to produce millions of RSA keys quickly and the standard way just doesn't cut it.

Here's yet another fast way to generate primes which has actually resisted years of review.

Challenge files:

- [fast_primes.py](#)
- [key.pem](#)
- [ciphertext.txt](#)

chipher text =

249d72cd1d287b1a15a3881f2bff5788bc4bf62c789f2df44d88aae805b54c9a94b8944c0ba79
8f70062b66160fee312b98879f1dd5d17b33095feb3c5830d28

source code chall :

```
#!/usr/bin/env python3

import math
import random

from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Util.number import bytes_to_long, inverse
from gmpy2 import is_prime

FLAG = b"crypto{????????????}"

primes = []

def sieve(maximum=10000):
    # In general Sieve of Sundaram, produces primes smaller
    # than (2*x + 2) for a number given number x. Since
    # we want primes smaller than maximum, we reduce maximum to half
    # This array is used to separate numbers of the form
    # i+j+2ij from others where 1 <= i <= j
    marked = [False]*(int(maximum/2)+1)

    # Main logic of Sundaram. Mark all numbers which
    # do not generate prime number by doing 2*i+1
```

```

    for i in range(1, int((math.sqrt(maximum)-1)/2)+1):
        for j in range(((i*(i+1)) << 1), (int(maximum/2)+1), (2*i+1)):
            marked[j] = True

    # Since 2 is a prime number
    primes.append(2)

    # Print other primes. Remaining primes are of the
    # form 2*i + 1 such that marked[i] is false.
    for i in range(1, int(maximum/2)):
        if (marked[i] == False):
            primes.append(2*i + 1)

def get_primorial(n):
    result = 1
    for i in range(n):
        result = result * primes[i]
    return result

def get_fast_prime():
    M = get_primorial(40)
    while True:
        k = random.randint(2**28, 2**29-1)
        a = random.randint(2**20, 2**62-1)
        p = k * M + pow(e, a, M)

        if is_prime(p):
            return p

sieve()

e = 0x10001
m = bytes_to_long(FLAGS)
p = get_fast_prime()
q = get_fast_prime()
n = p * q
phi = (p - 1) * (q - 1)
d = inverse(e, phi)

key = RSA.construct((n, e, d))

```

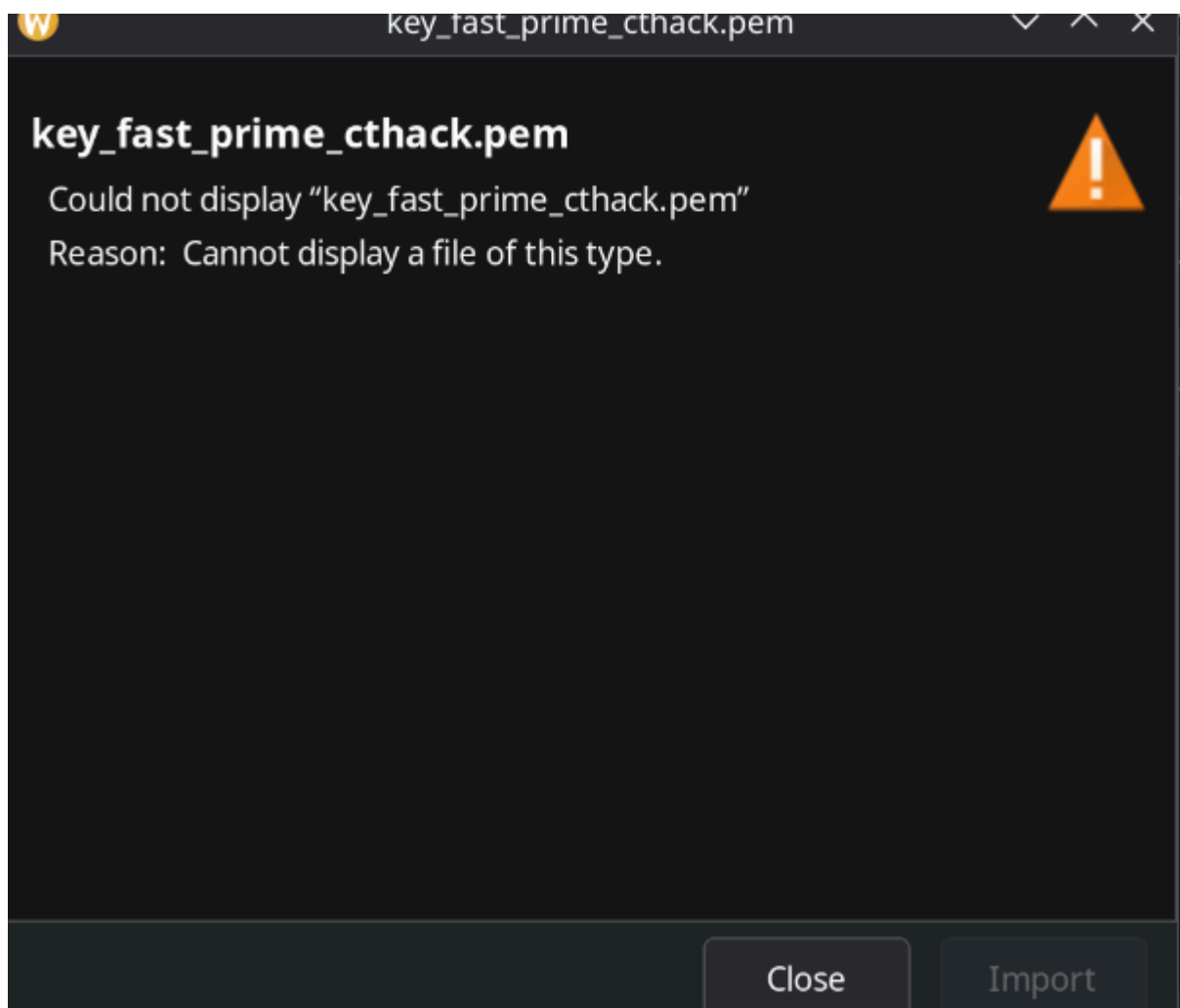
```
cipher = PKCS1_OAEP.new(key)
ciphertext = cipher.encrypt(FLAG)

assert cipher.decrypt(ciphertext) == FLAG

exported = key.publickey().export_key()
with open("key.pem", 'wb') as f:
    f.write(exported)

with open('ciphertext.txt', 'w') as f:
    f.write(ciphertext.hex())
```

di deskripsi chall kita di kasih key.pem dimana file ini ketika ku buka malah eror karena untuk mendapat nilai nya kita harus mengimportnya dulu.



cara mengetahui nilai dari kunci ini kita bisa pakai command ssl

bash :

openssl rsa -in key.pem -text -noout -pubin

output =

```
pinn@pinn:/run/media/pinn/New Volume/py$ openssl rsa -in key_fast_prime_ckptack.pem -text
Could not find private key from key_fast_prime_ckptack.pem
00A360A9197F0000:error:1608010C:STORE routines:ossl_store_handle_load_result:unsupported:crypto/store/store_result
.c:160:provider=default
pinn@pinn:/run/media/pinn/New Volume/py$ openssl rsa -in key_fast_prime_ckptack.pem -text -pubin
Public-Key: (511 bit)
Modulus:
  4c:a2:1e:de:37:e3:d6:a6:3b:ce:e5:f9:12:0d:09:
  89:f0:14:3a:ae:91:53:02:f3:46:e8:ab:bb:84:0a:
  72:fb:56:d1:48:75:28:ca:75:19:5b:78:32:62:83:
  89:09:2c:4e:89:3f:a4:a6:c9:b0:11:42:66:e8:05:
  b1:2d:c0:8d
Exponent: 65537 (0x10001)
writing RSA key
-----BEGIN PUBLIC KEY-----
MFswDQYJKoZIhvcNAQEBBQADSwAwRwJATKIE3jffj1qY7zuXSEg0JifAUQq6RUwLz
RuiRu4QKcvtW0Uh1Kmp1GVt4MmKDiQksTok/pKbJsBFCZugFsS3AjQIDAQAB
-----END PUBLIC KEY-----
pinn@pinn:/run/media/pinn/New Volume/py$
```

dari source code kita bisa tau bahwa flag kita sebelum di buat jadi ciphertext di padding menggunakan PKCS1_OAEP,baru text hasil padding tsb di tulis ke ciphertext.txt

```
key = RSA.construct((n, e, d))
cipher = PKCS1_OAEP.new(key)
ciphertext = cipher.encrypt(FLAG)
```

Solusi

jadi karena n bisa kita faktor kan di factordb untuk mendapat nilai dari p dan q maka kita tinggal menghitung chall RSA ini seperti biasa.ketika n,e,d,p,q sudah kita dapat maka kita harus menghilangkan padding dari PKCS1_OAEP dengan membuat variable key_private yang berisi n,e,d,p,q untuk kita buat jadi format key private yang di terima objek PKCS1_OAEP.setelah itu baru kita bisa mendapatkan flag dengan men-decrypt c dari ciphertext menggunakan objek PKCS1_OAEP.

solver =

```
from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Util.number import long_to_bytes
def get_n_dan_e():
    with open("key_fast_prime_ckptack.pem","r") as f:
```

```

        key =RSA.import_key(f.read())
        return key.n , key.e

n,e = get_n_dan_e()
print(n)
c =
bytes.fromhex("249d72cd1d287b1a15a3881f2bff5788bc4bf62c789f2df44d88aae8
05b54c9a94b8944c0ba798f70062b66160fee312b98879f1dd5d17b33095feb3c5830d2
8")
p =
51894141255108267693828471848483688186015845988173648228318286999011443
419469
q =
77342270837753916396402614215980760127245056504361515489809293852222206
596161
print(p * q == n)

phi = (p-1)*(q-1)
d = pow(e,-1,phi)

key_param = (n,e,d,p,q)
private_key = RSA.construct(key_param)
print(private_key)

try:
    cipher = PKCS1_OAEP.new(private_key)
    decrypted = cipher.decrypt(c)
    print(f"[+] Flag berhasil didapat: {decrypted}")
except ValueError as err:
    print(f"[-] Gagal dekripsi dengan OAEP default (SHA-1): {err}")
    print("[*] Mencoba dengan Hash Algorithm lain (SHA256)...")

# Kadang soal CTF memakai SHA-256 sebagai hash function OAEP

```